

Uninformed Search Algorithms

1. Breadth First Search (BFS) ✓
2. Depth First Search (DFS) ✓
3. Iterative Deepening Depth First Search (IDDFS) ✓

Informed Search Algorithms

1. Greedy Best First Search ✓
2. A* Search ✓

Local Search / Optimization Algorithms

1. Hill Climbing Algorithm ✓
2. CSP Backtracking Algorithm ✓
3. Genetic Algorithm (GA) ✓

Adversarial Search Algorithms

1. Minimax Algorithm ✓
2. Alpha-Beta Pruning ✓

Problem Formulation

1. Initial State
2. Action
3. Transition model
4. Goal Test
5. Path cost function

Eg: Map of Romania

Initial State : In(Arad)

Action : if current state is In(Arad),
actions = $\{ \text{Go(Sibiu)}, \text{Go(Timisoara)}, \text{Go(Zerind)} \}$

Transition Model :

Results (In(Arad), Go(Sibiu)) = In(Sibiu)

Goal Text :

/
explicit

clearly defined GS

Eg: In(Bucharest)

implicit

not rep. by a fixed state

Eg: checkmate (chess)

Path cost function: assign numeric cost to each path.

$$\text{step cost} = c(x, a, y)$$

↑
action

$$c(\text{Arad}, \text{Go}(\text{Sibiu}), \text{Sibiu})$$

eg: Vacuum cleaner

Initial state: Room A/B

Actions: Right, left, suck

Transition model:

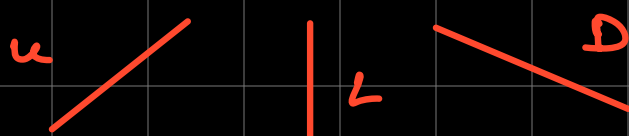
$$\text{Result}(\text{In}(\text{Room A}), \text{Suck}) = \text{Clean}(\text{A})$$

Path cost:

1

8-puzzle

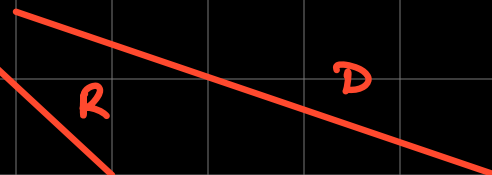
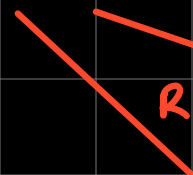
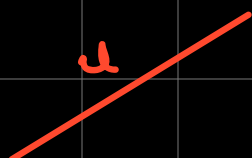
1	2	3
	4	6
7	5	8



1	2	3
7	4	6
	5	8

1	2	3
4		6
7	5	8

	2	3
1	4	6
7	5	8



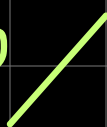
1	2	3
4	5	6
7		8

1	2	3
4	6	
7	5	8

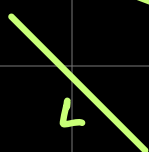
1	2	3
	4	6
7	5	8

1		3
4	2	6
7	5	8

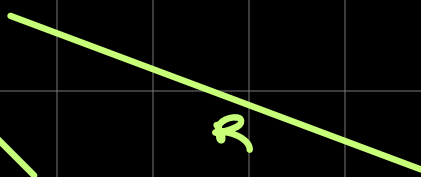
D



L



R



1	2	3
4		6
7	5	8

1	2	3
4	5	6
7	8	

1	2	3
4	5	6
	7	8

Initial State : Given any board with nos 1 to 8
and one empty slot

Action : blank moves L, R, U, D

Transition model : state + action $\Rightarrow$ result
state

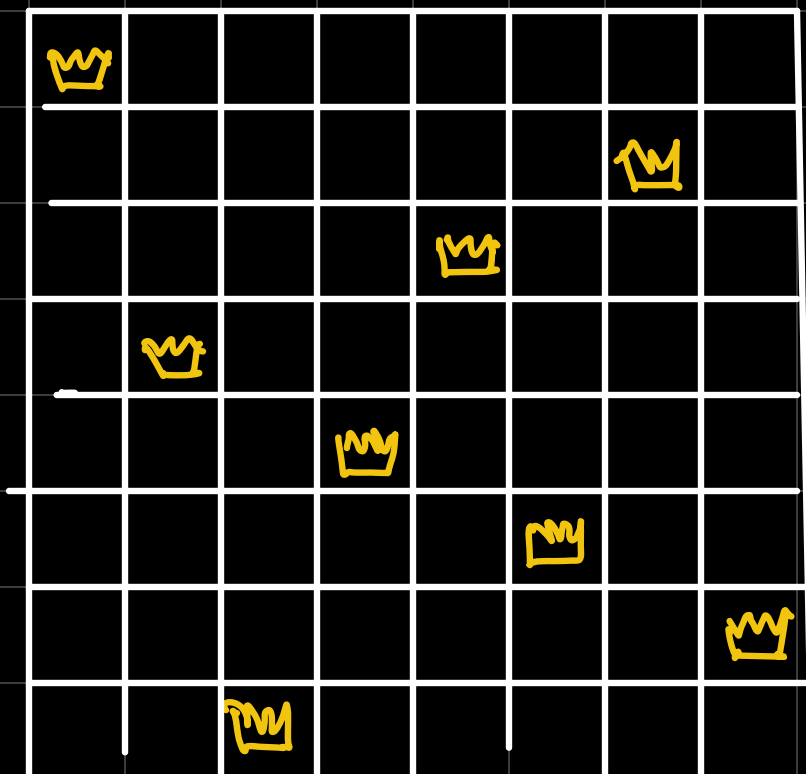
eg:

Goal Test : Board reaches needed layout
(explicit)

Path cost : step cost 1

N-Queen Problem

Place N Queens such that no two of them are in the same row/column, or diagonal to each other.



State: 0 to N Queens on board

Initial state: Empty board

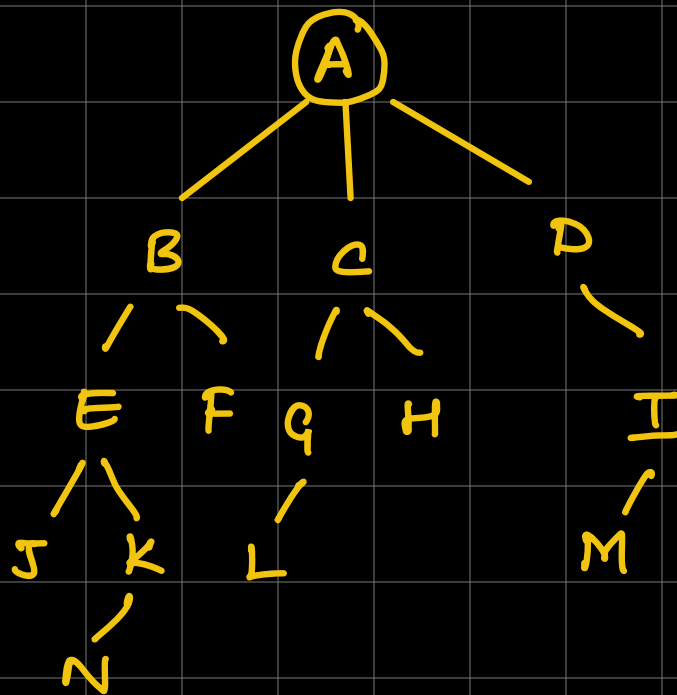
Transition model: Board added with a queen
keeping in check all the conditions.

Goal Test: N Queens on board none attacked

Uninformed Search

↳ no prior knowledge

1. BFS



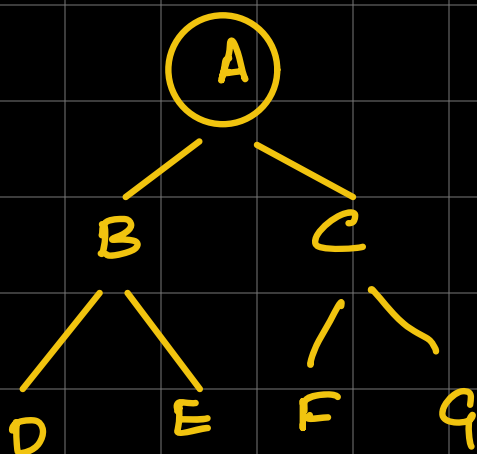
A → B → C → D
→ E → F → G
→ H → I →
J → K → L →
M → N

Queue: A ✓ B ✓ C ✓ D ✓ E ✓ F ✓ G ✓ H ✓ I ✓ J ✓ K ✓ L ✓
N ✓

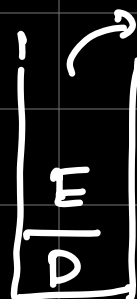
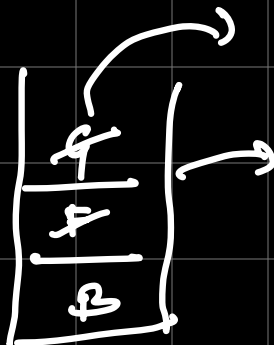
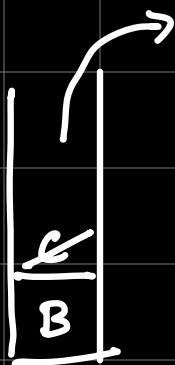
TC: $O(V+E)$

2. DFS

$A \rightarrow C \rightarrow G \rightarrow F \rightarrow B$
 $\rightarrow E \rightarrow D$



Stack



$O(b^d)$
↑
branching
factor

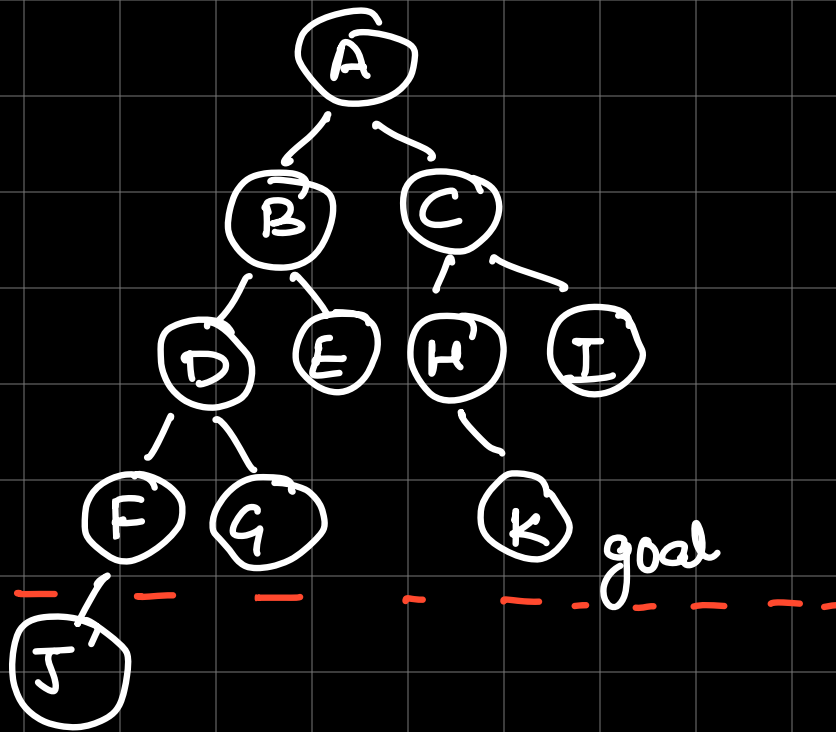
↖ depth

3. DLS

variant of DFS that imposes a depth on the search tree. This limit helps prevent the search from going too deep. $\Rightarrow$ getting stuck ∞ loop

$\Rightarrow$ Useful when search space is infinite

limit = 3



$A \rightarrow B \rightarrow D \rightarrow F \rightarrow G \rightarrow E \rightarrow C \rightarrow H \rightarrow K$

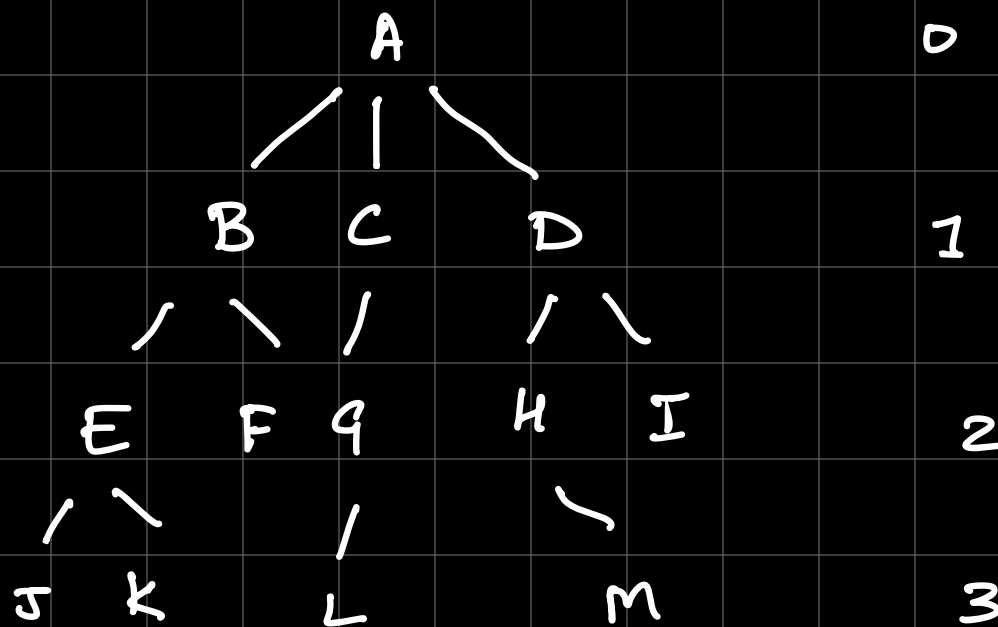
$A \rightarrow C \rightarrow I \rightarrow H \rightarrow K$

$\Rightarrow A \rightarrow C \rightarrow H \rightarrow K$

TC: $O(b^2)$

4. IDDFS

- combines benefits of BFS + DFS
- searches to a limited depth $\Rightarrow$ increasing limit with each iteration
- Ensure completeness + optimality
- ensures all nodes at each depth b4 going deeper.



It 0:

A

It 1:

$A \rightarrow B \rightarrow C \rightarrow D$

It 2:

$A \rightarrow B \rightarrow E \rightarrow F \rightarrow C \rightarrow G \rightarrow D \rightarrow H \rightarrow \underline{\underline{I}}$

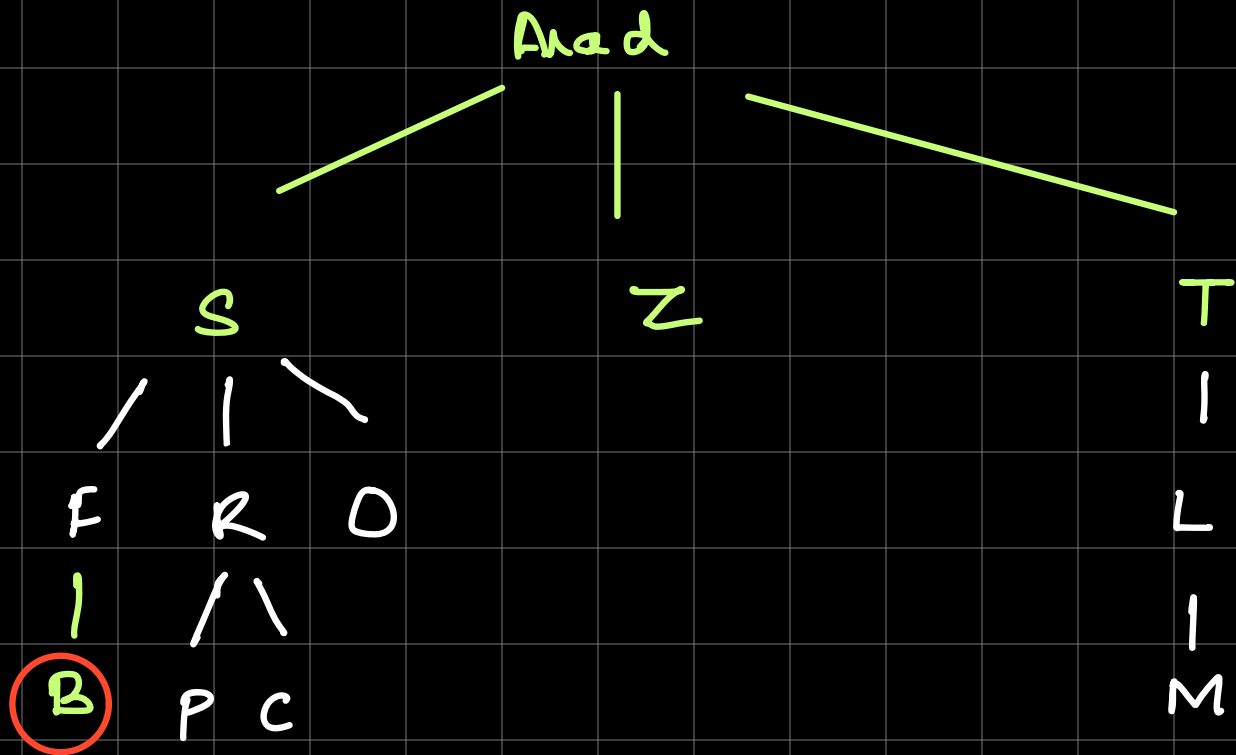
^x
It 3:

$A \rightarrow B \rightarrow E \rightarrow J \rightarrow K \rightarrow F \rightarrow C \rightarrow G \rightarrow L \rightarrow D \rightarrow$
 $H \rightarrow M \rightarrow \underline{I}$

Romania Problem

A S I T F G P L B D
C M

BFS:



DFS

$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$

$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$

$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$

$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$

Asad

/

S

/

R

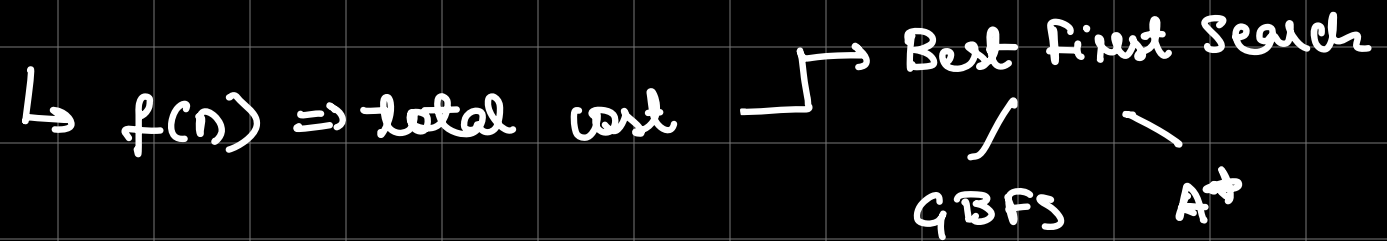
/

P

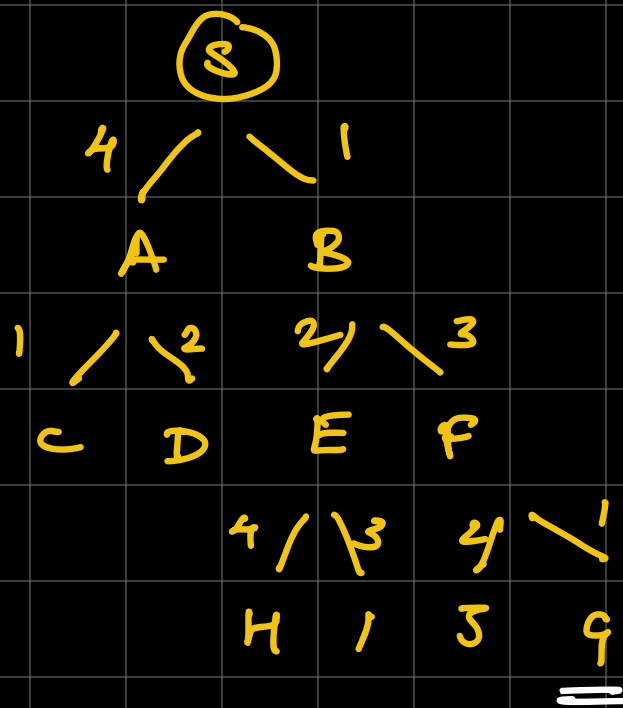
I

B

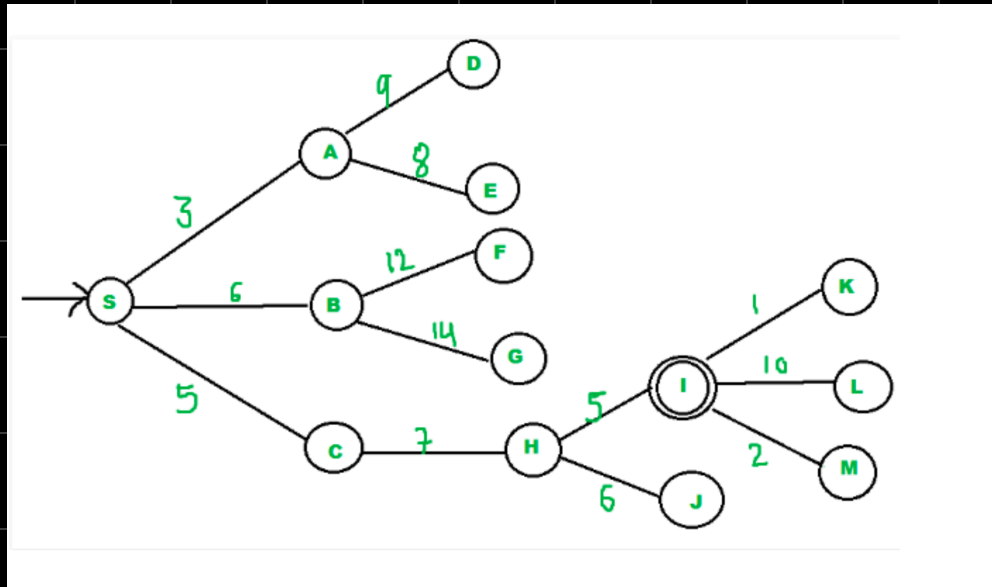
Informed Search



Best first Search



S → B → F → G



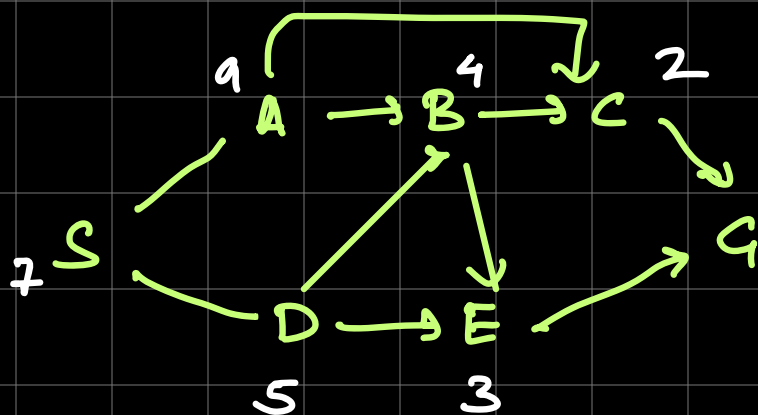
Best First

$S \rightarrow C \rightarrow H \rightarrow I$

GBFS: $f(n) = h(n)$

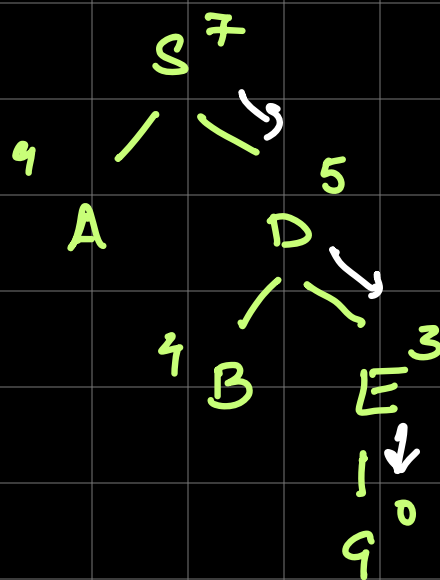
↑ closeness to goal

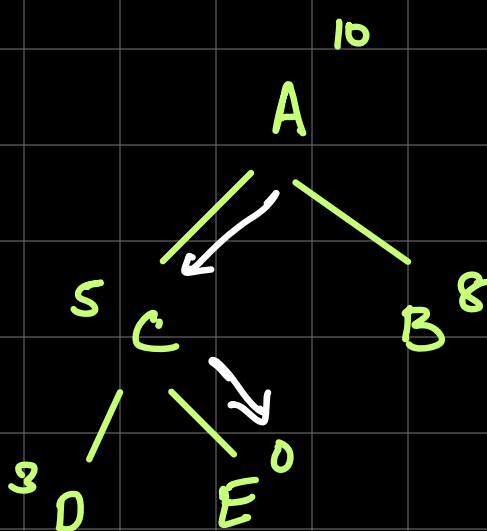
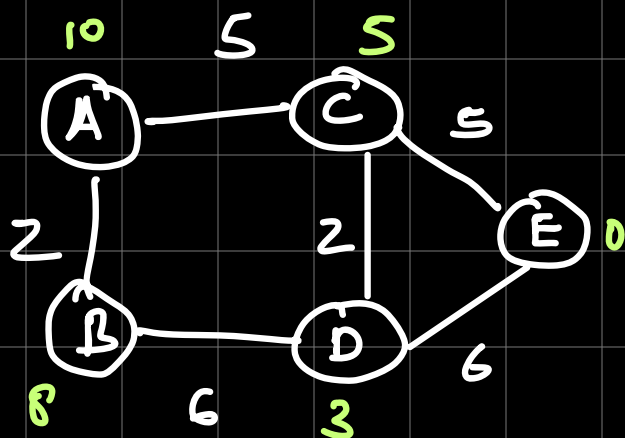
Heuristic function informs the search abt.
direction to the goal.



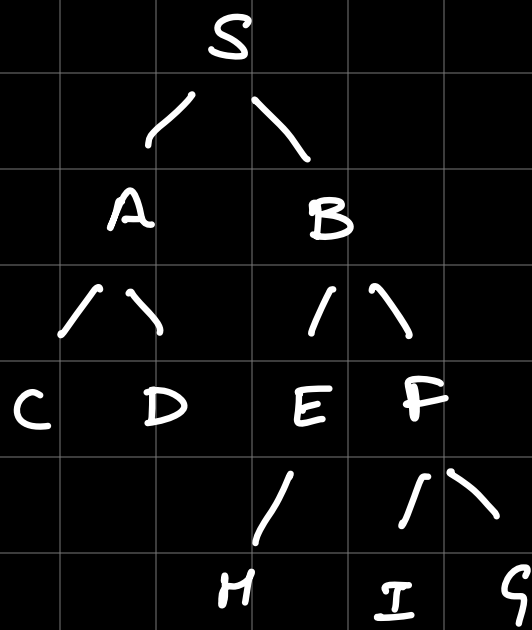
$$f(n) = h(n)$$

→



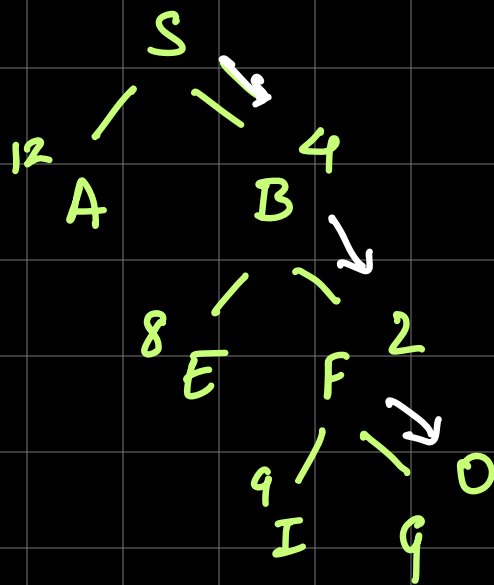


$$f(n) = h(n)$$



A	12
B	4
C	7
D	3
E	8
F	2
H	4
I	9
S	13
G	0

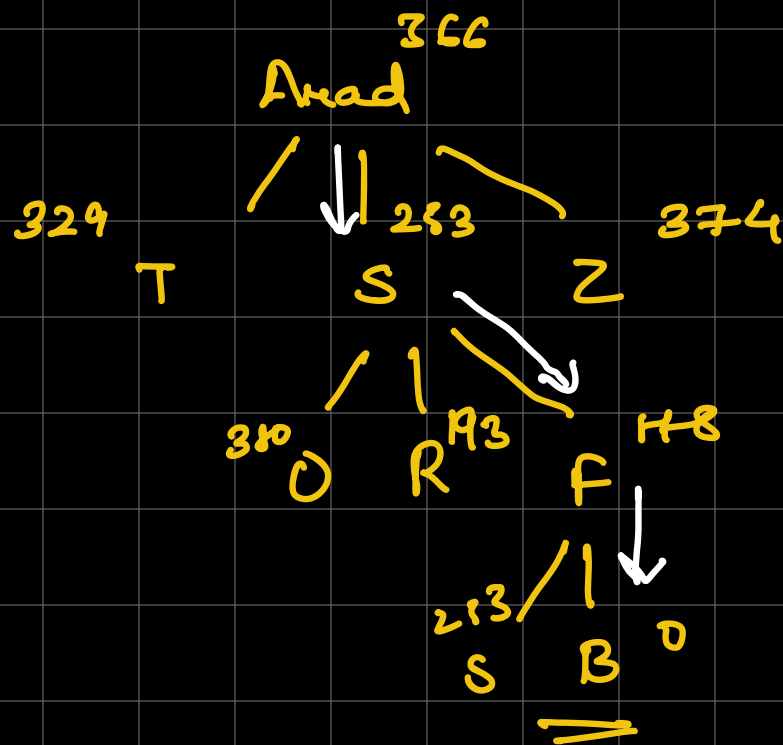
$$f(n) = h(n)$$



Map of Romania

GBFS:

$$f(n) = h(n)$$



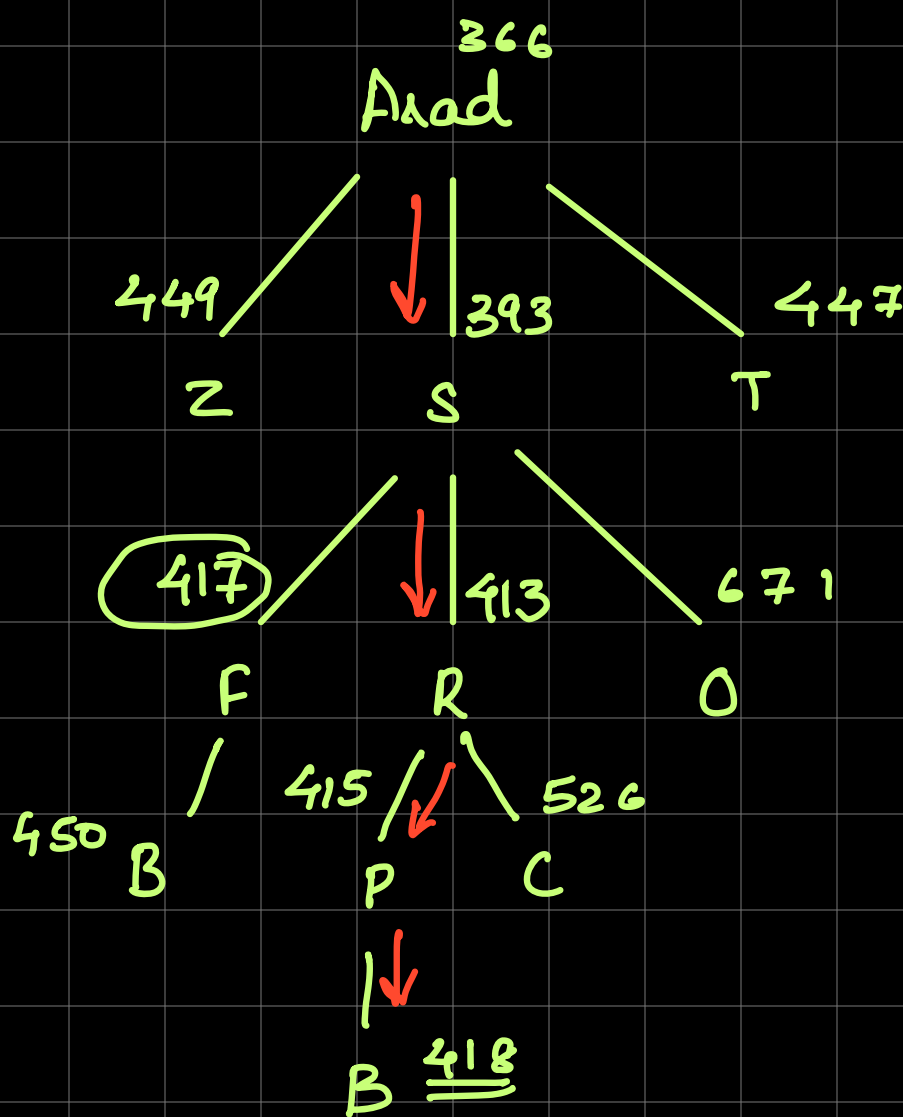
$A \rightarrow S \rightarrow F \rightarrow B$

A*

$$f(n) = g(n) + h(n)$$

↳ path
to n

↳ heuristic



Admissibility and Consistency

1) $h(n)$ = admissible heuristic

✓

?

one that never overestimates the cost to reach the goal

2) $g(n)$ → cost of path to reach n along current path.

$$h(n) \leq h^*(n)$$

↳ estimated cost

ADMISSIBILITY

Heuristic for 8-puzzle

$h_1(N)$ = no. of misplaced tiles

$h_2(N)$ = manhattan dist.

(18)

8	3	5
4	1	2
2	7	

$$h_1(N) = 6$$

$$h_2(N) = 3 + 1 + 2 + 0 + 2 + 3 + 1 = 12$$

N-Queens

heuristic

$h(n)$ = no. of attacking
Q. pairs

Graph Colouring
heuristic

$h(n)$ = no. of adj
node pairs
having same col.

Local Search Algorithms:

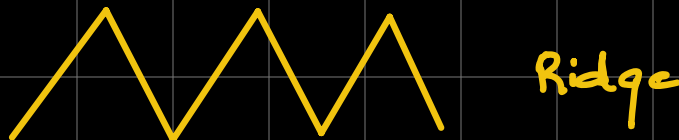
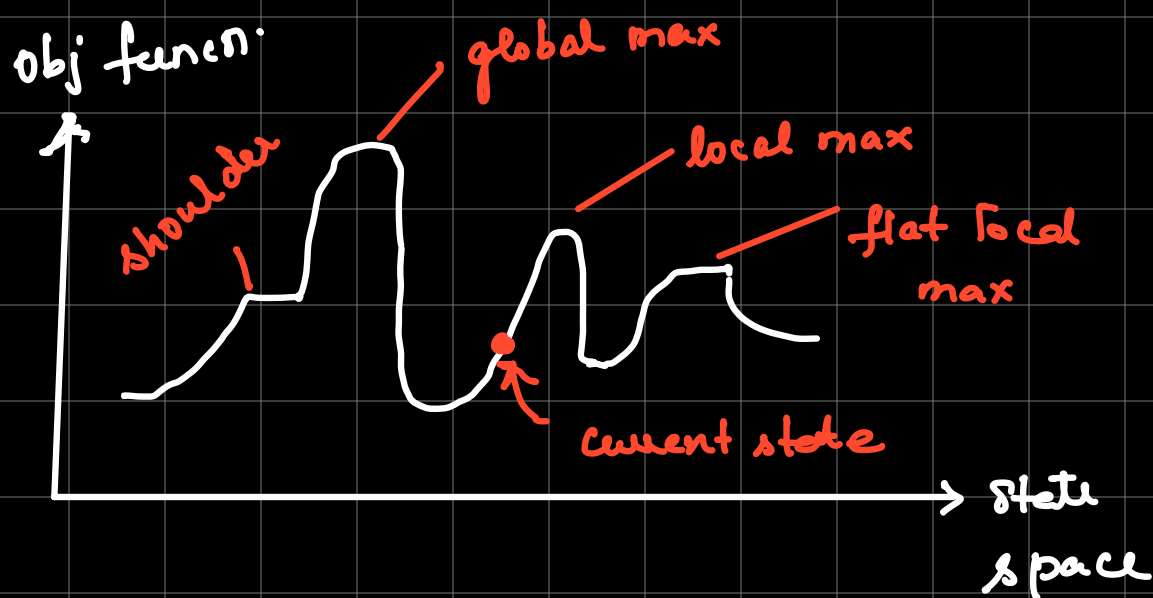
→ heuristic method of solving computationally hard opt. problems

→ moves soln. → soln in search space by applying local changes till optimal deemed or time bound elapsed.

not
syst.
Adv

little memory

find reasonable soln. in large state spaces.



Local max: state better than neighbour states however there exist a better state $\Rightarrow$ G.M.

Global maximum: best possible state in the state space diagram

Plateau / FLM: flat region where neighbours have same heuristic value.

Ridge: special kind of local max. better than neighbours but itself has a slope

shoulder: plateau with uphill edge

Hill Climbing Problem

1. Define current state = initial state

2. loop until goal is ach. or no more operation on current state:

→ Apply op. to get new state

→ Compare new → goal

→ "Quit" if = goal

→ Evaluate new state $h(n)$ & compare to current state

→ If closer to goal then update current state

Disadv. → local maxima → global max exists which can't be reached
→ plateau → neighbours are all equal so terminates without
→ Ridge ⇒ no backtracking greedy

⇒ follows an uphill path, always choosing the best possible neighbour till it reaches a peak.
↳ greedy

⇒ does not maintain a space tree ⇒ no memory beyond current state
∴ no backtracking

Constraint Satisfaction Problem (CSP)

Goal → Discover some problem state that satisfies a given set of constraints

State → defined by variables x_i from Domain D_i

4-Queens

Variables Q_1, Q_2, Q_3, Q_4

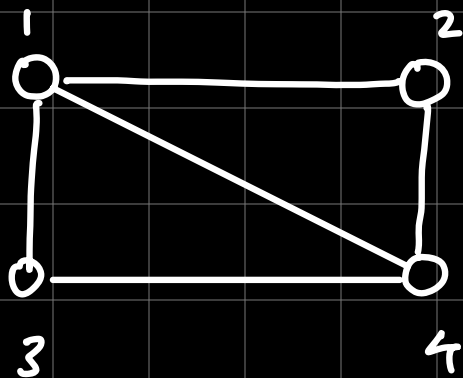
Domain $D_i = \{1, 2, 3, 4\}$

Constraint

$$Q_i \neq Q_j$$

$$|Q_i - Q_j| \neq |i - j|$$

CSP $\Rightarrow$ Backtracking smart



$$V = \{1, 2, 3, 4\}$$

$$D = \{R, G, B\}$$

$$C = \{1 \neq 2, 1 \neq 3, 1 \neq 4, 2 \neq 4, 3 \neq 4\}$$

	1	2	3	4
Initial Dom	RGB	RGB	RGB	RGB

	1	2	3	4
1 = R	R	GB	GB	GB

	1	2	3	4
2 = G	R	G	GB	B

	1	2	3	4
3 = B	R	G	B	B

empty conflict

	1	2	3	4
3 = G	R	G	G	B

Crypt Arithmetic

Carry $\Rightarrow$ can be 0 or 1

$$\begin{array}{r} T O \\ + 9 O \\ \hline O V T \end{array}$$

Assign no. to letters (0-9)
no 2 letters have same val
only 1 carry

$$\therefore O = 1$$

$\nearrow$ carry generate
8 +

$$\begin{array}{r} 2 1 \\ + 8 1 \\ \hline 1 0 2 \end{array}$$

$\text{carry} = 0/1$
 $\begin{array}{r} \text{T} \text{ W} \text{ O} \\ \text{T} \text{ W} \text{ O} \\ \hline \text{F} \text{ O} \text{ U} \text{ R} \end{array}$

$\therefore F = 1$

now for $T+T$ to generate
 carry T has ≥ 5
 by $T \neq 5$ because
 then '0' = 0

$\therefore$ assume $T = 6$

6	2
6	2
1 2	4

now W 2 0
 $\hookrightarrow 3, 5, 7, 8, 9$

$\hookrightarrow 3 + 3 = 6$ but $T = 6$
 lets take $W = 5$

but beyond ≥ 5 violates
 '0' value

$\therefore$ change T

$$T = 7$$

$$\begin{array}{r} 734 \\ 734 \\ \hline 1468 \end{array}$$

$$\Rightarrow F = 1$$

$$T = 7$$

$$O = 4$$

$$W = 3$$

$$U = 6$$

$$U_i = \{T, W, O, F, U, R\}$$

$$D_i = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$$

Constraints $\rightarrow$ ① No 2 letter same val

② Arithmetic addn. should hold

③ No more than 1 car

conv

	B	A	S	E
	B	A	L	L
9	A	M	E	S

$B \geq 5$

$B \neq 5$

$A \neq 0$

B
A
S
E
L
G
M

		1		
	7	4	8	3
	7	4	5	5
1	4	9	3	8

3, 5, 6, 8, 9

SEND
MORE

MONEY

M = 1

9567
1085

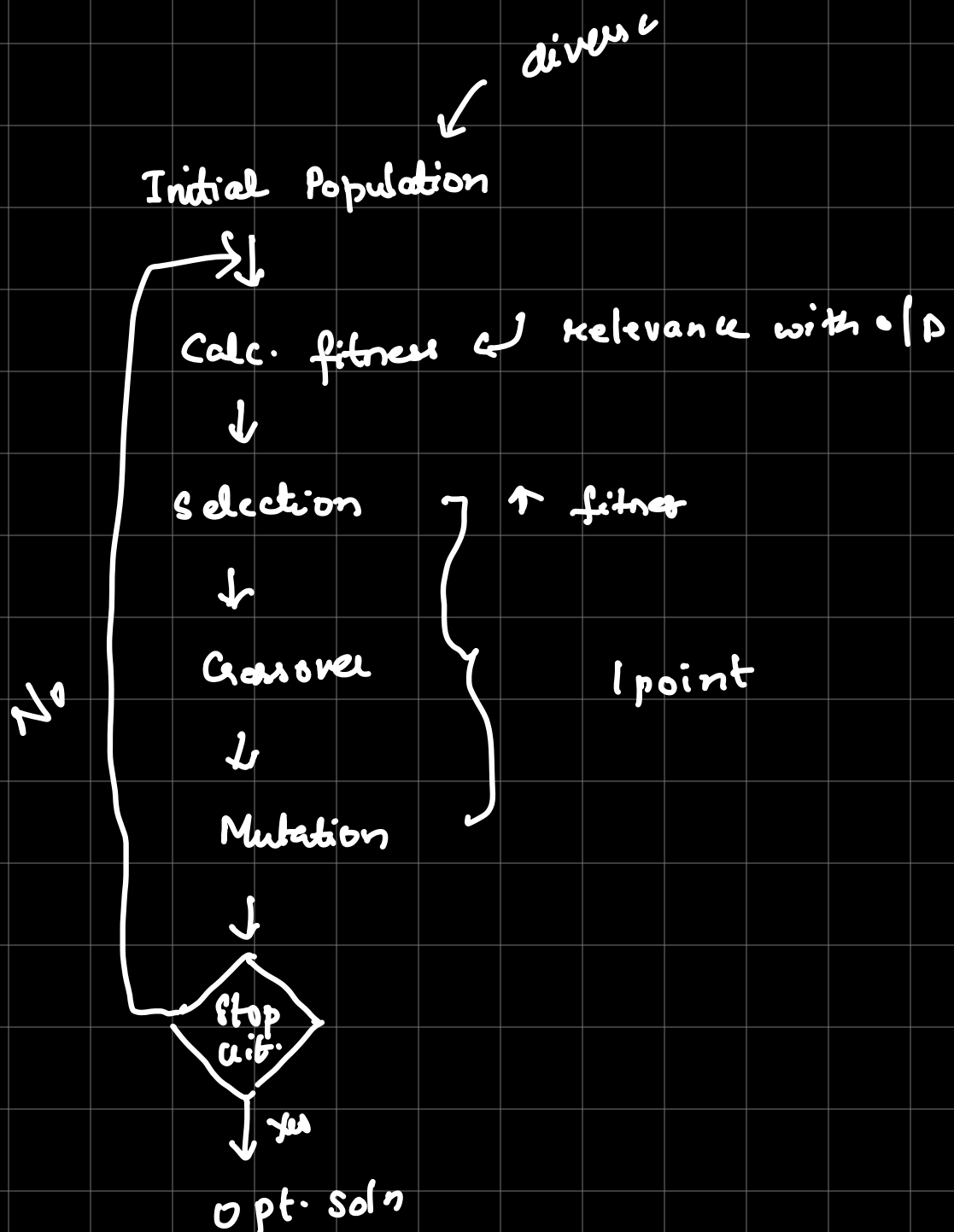
10652

S
E
N
D
M
O
R
Y

= 1
= 0

Genetic Algorithm

- ↳ abstraction of real bio. evolution
- ↳ solve complex problems (NP hard)
- ↳ focus on optimization



0/1 Knapsack

Item	1	2	3	4	5	6	7
p_i	6	5	8	9	6	7	3
w_i	2	3	6	7	5	9	4

$$W = 9$$

1. Chromosome Representation
 $\Rightarrow$ encoding of soln.

1 = item selected 0 = x

2. Initial Population

Generate random candidate soln.

$C_1 =$ 1 1 0 0 0 0 1

$C_2 =$ 1 0 1 1 0 0 0

$C_3 =$ 0 1 1 0 1 0 0

$C_4 =$ 1 0 0 1 0 1 0

3. Fitness Function

↳ quality of soln. wrt o/p.

$$\text{Fitness} = \begin{cases} \sum p_i x_i & ; \sum w_i x_i \leq 9 \\ 0, & \text{otherwise} \end{cases}$$

$$C_1 = 14$$

$$C_2 = 0 > 9$$

$$C_3 = 0$$

$$C_4 = 0$$

Step 4: Selection $C_1 \& C_2$
 steps:
 1. crossover

$$\begin{array}{cccc|cccc} 1 & 1 & 0 & 0 & 1 & 0 & 0 & 1 \\ 1 & 0 & 1 & 1 & 1 & 0 & 0 & 0 \end{array} \Rightarrow \begin{array}{cccc|cccc} 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 & 0 & 0 & 1 \end{array}$$

Step 6 : Mutation

1	1	0	0	1	0	0
1	0	1	0	0	0	1

w_i	$p_i x_i$
$\Rightarrow 10$	0
	0

Graph

$V \Rightarrow A, B, C, D, E$

$C \Rightarrow 1, 2, 3$

Adversarial Search Algorithm

Min - Max

max : best move

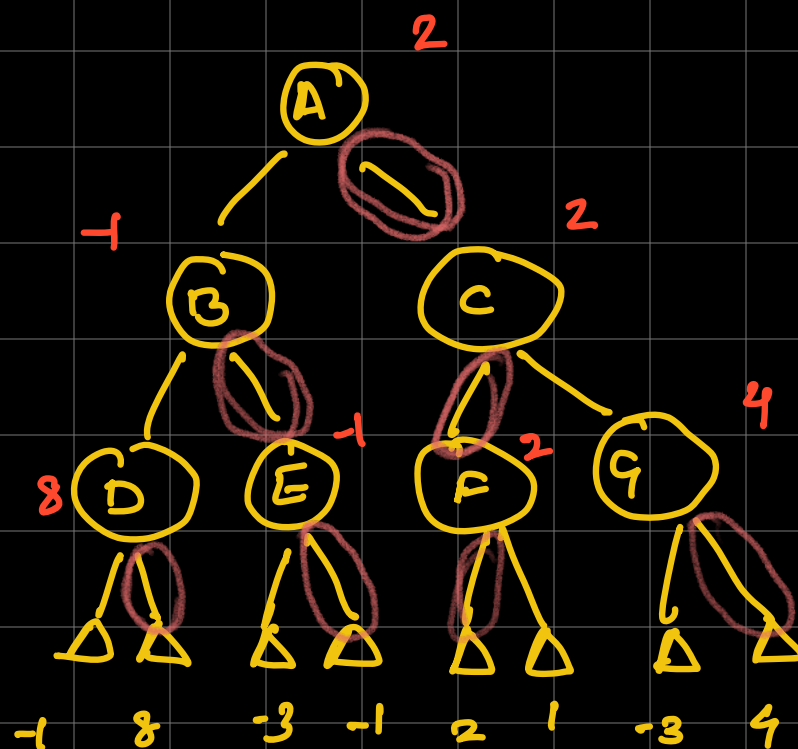
min : worst move wrt me

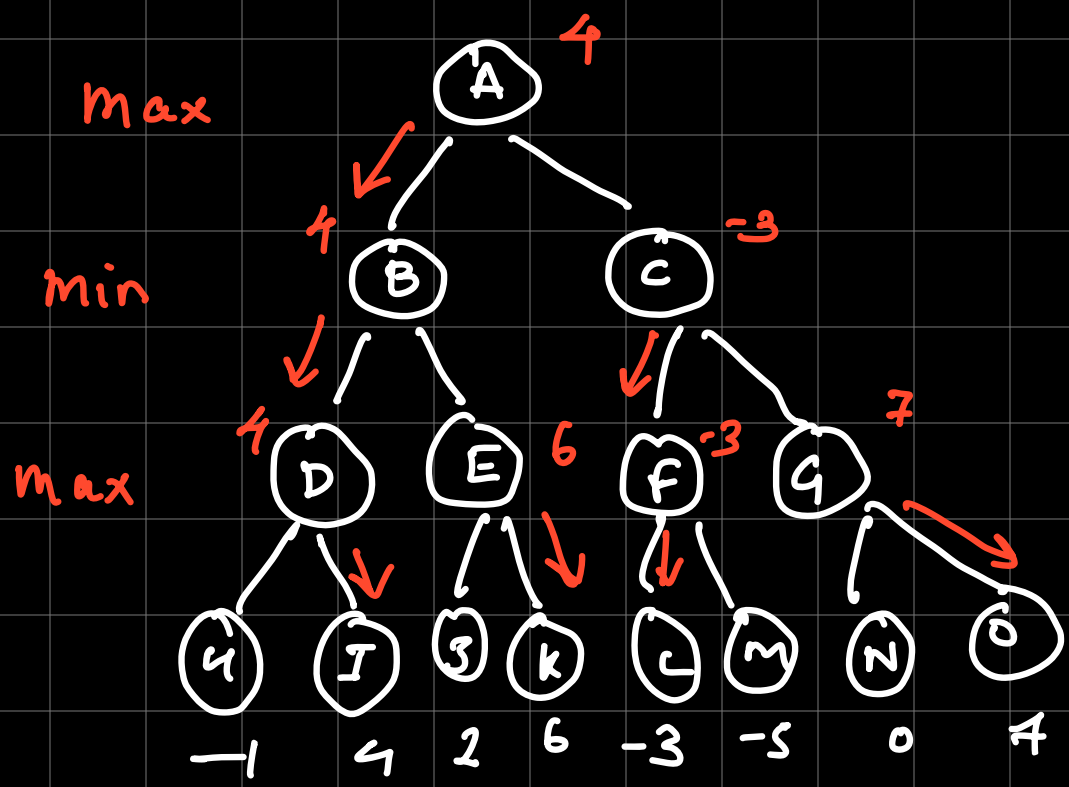
$O(b^d)$

max

min

max





Alpha - Beta Pruning

$$\alpha = \max$$
$$\beta = \min$$

↳ min max $O(b^d)$

↳ cut off search by exploring less nodes

$$\alpha \geq \beta$$

⇒ prune

